

UNIVERSITY OF ENGINEERING AND TECHNOLOGY PESHAWAR

Department of Electrical Engineering

LABORATORY REPORT

Subject:	EE-170L Computer Programming Lab
Lab No:	Lab 11
Lab Title:	Strings in C++
Semester:	Semester 2
Session:	Spring 2026

Submitted By:

Name: Muhammad Adil

Registration No: BF25NWELE0669

1. OBJECTIVE

To learn and apply C-strings and string objects in C++ including declaration, initialization, input/output operations, and string manipulation functions.

2. THEORY

Strings in C++ are collections of characters. There are two types: (1) C-Strings (C-style Strings): Arrays of type char terminated with null character '\0'. Fixed size, manual memory management. Input using cin (stops at space) or cin.get() (reads line). Functions include strcpy(), strcat(), strlen(), strcmp(), strrev(), strlwr(),strupr(). (2) String Objects (C++ Standard Library): Objects of string class with dynamic size and automatic memory management. Input using getline(cin, str) which reads entire lines including spaces.

Key String Functions: strcpy(s1, s2) copies s2 into s1; strcat(s1, s2) concatenates s2 to end of s1; strlen(s) returns length; strcmp(s1, s2) compares two strings; strrev(s) reverses; strlwr(s) converts to lowercase;strupr(s) converts to uppercase.

3. LAB TASKS

3.1 String Length Calculation

Write a C++ program that declares and initializes a string variable called 'message' with a phrase of your choice. Use a built-in function to calculate and display the length of the string.

Source Code:

```
#include <iostream>
#include <cstring>      // Header file for C-string functions
using namespace std;

int main()
{
    // =====
    // String Declaration and Initialization
    // =====
    // Initialize a C-string with a phrase
    // The compiler automatically adds null character '\0' at the end
    char message[] = "Welcome to C++ Programming Lab!";

    // Alternative declarations:
    // char message[50] = "Welcome to C++ Programming Lab!";
    // char message[50] = {'W','e','l','c','o','m','e','\0'};

    // =====
    // Calculate String Length
    // =====
    // strlen() function returns the number of characters
    // It counts characters excluding the null terminator '\0'
    int length = strlen(message);
```

```

// =====
// Display Results
// =====
cout << "===== STRING LENGTH =====" << endl;
cout << "Original String: " << message << "" << endl;
cout << "Length of string: " << length << " characters" << endl;
cout << "===== " << endl;

// Demonstrate with another string
char another[] = "Muhammad Adil";
cout << "\nString: " << another << "" << endl;
cout << "Length: " << strlen(another) << " characters" << endl;

return 0;
}

```

Expected Output:

```

===== STRING LENGTH =====
Original String: "Welcome to C++ Programming Lab!"
Length of string: 29 characters
=====

String: "Muhammad Adil"
Length: 13 characters

```

3.2 String Concatenation

Declare two string variables called 'firstName' and 'lastName' and initialize them with your first and last name. Concatenate the two strings to form a full name and display it.

Source Code:

```

#include <iostream>
#include <cstring>    // Header file for C-string functions
using namespace std;

int main()
{
    // =====
    // String Declaration and Initialization
    // =====
    // Declare strings with sufficient size for concatenation
    // Size must be large enough to hold both strings + null character
    char firstName[50] = "Muhammad";    // First name
    char lastName[50] = "Adil";        // Last name
    char fullName[100];                // Buffer for full name

```

```

// =====
// Method 1: Using strcpy and strcat functions
// =====
// strcpy() - Copies string to destination (including null terminator)
// strcat() - Concatenates (appends) string to destination

strcpy(fullName, firstName);    // Copy firstName to fullName
strcat(fullName, " ");          // Add space between names
strcat(fullName, lastName);     // Append lastName to fullName

// =====
// Display Results using C-strings
// =====
cout << "===== STRING CONCATENATION =====" << endl;
cout << "First Name:  " << firstName << endl;
cout << "Last Name:   " << lastName << endl;
cout << "Full Name:    " << fullName << endl;
cout << "===== " << endl;

// =====
// Method 2: Using C++ string objects
// =====
// string class allows easy concatenation using + operator
string fname = "Muhammad";
string lname = "Adil";
string fullname = fname + " " + lname; // Using + operator

cout << "\nUsing C++ string objects:" << endl;
cout << "Full Name: " << fullname << endl;

return 0;
}

```

Expected Output:

```

===== STRING CONCATENATION =====
First Name:  Muhammad
Last Name:   Adil
Full Name:   Muhammad Adil
=====

Using C++ string objects:
Full Name: Muhammad Adil

```

3.3 Demonstration: String Input Methods

Demonstrate different methods of string input in C++. The cin operator stops reading at whitespace, cin.get() reads a line up to a specified length, and getline() for string objects reads the entire line including spaces.

Source Code:

```
#include <iostream>
#include <string>      // Header for C++ string class
using namespace std;

int main()
{
    // =====
    // Method 1: Using cin (stops at space)
    // =====
    char str1[100];
    cout << "Enter a string (cin - stops at space): ";
    cin >> str1; // Reads until whitespace is encountered
    cout << "You entered: " << str1 << endl;
    cin.ignore(); // Clear input buffer to remove newline

    // =====
    // Method 2: Using cin.get() (reads full line)
    // =====
    char str2[100];
    cout << "\nEnter a string (cin.get - reads full line): ";
    cin.get(str2, 100); // Reads up to 99 characters or newline
    cout << "You entered: " << str2 << endl;
    cin.ignore(); // Clear input buffer

    // =====
    // Method 3: Using C++ string with getline()
    // =====
    string str3;
    cout << "\nEnter a string (getline - reads full line): ";
    getline(cin, str3); // Reads entire line including spaces
    cout << "You entered: " << str3 << endl;

    // =====
    // Demonstrate String Functions
    // =====
    char sample[] = "Hello World";
    cout << "\n===== STRING FUNCTIONS =====" << endl;
    cout << "Original: " << sample << "\"" << endl;
    cout << "Length: " << strlen(sample) << endl;

    // Copy demonstration using strcpy
    char copy[50];
    strcpy(copy, sample);
    cout << "Copy: " << copy << "\"" << endl;
```

```

        // Concatenation demonstration using strcat
        strcat(copy, "!!!");
        cout << "After strcat: " << copy << " " << endl;

        return 0;
    }

```

Expected Output:

```

Enter a string (cin - stops at space): Electrical Engineering
You entered: Electrical

```

```

Enter a string (cin.get - reads full line): Electrical Engineering
You entered: Electrical Engineering

```

```

Enter a string (getline - reads full line): University of Engineering and
Technology
You entered: University of Engineering and Technology

```

```

===== STRING FUNCTIONS =====
Original: "Hello World"
Length: 11
Copy: "Hello World"
After strcat: "Hello World!!!"

```

4. CONCLUSION

This lab covered string handling in C++. We explored C-strings (char arrays) and C++ string objects. C-strings require manual handling and use functions like `strcpy()`, `strcat()`, and `strlen()` from the `cstring` header. The `cin` operator only reads up to whitespace, while `cin.get()` and `getline()` can read entire lines including spaces. C++ string objects offer easier manipulation with operators like `+` for concatenation and automatic memory management. Understanding both approaches is important for effective C++ programming, as C-strings are still widely used in system programming and legacy code.